High Performance Computing

MapReduce Assignment

Environment Setup

In this exercise, we will simulate the main stages of MapReduce using Python and standard command-line tools. A Hadoop cluster is not required.

You need:

- Python 3;
- a Linux, macOS, or WSL terminal;
- the standard `sort` command.

Linux / Ubuntu / WSL

Check that Python is installed:

```
python3 --version
```

If Python is not installed:

```
sudo apt update
sudo apt install python3
```

Windows users are recommended to use the Ubuntu/WSL environment that was also used for the MPI exercise.

macOS

Check Python using:

```
python3 --version
```

If necessary, Python can be installed using Homebrew:

```
brew install python
```

Objectives

The purpose of this exercise is to understand the basic MapReduce programming model and its data flow. You will implement mapper and reducer programs, observe how intermediate key–value pairs are grouped during the shuffle and sort stage, perform word counting, investigate the role of a combiner and partitioner, and implement a simple inverted index. You will also relate the practical exercises to scheduling, data locality, fault tolerance, and distributed storage concepts discussed in the lecture.

Part 1: Understanding Map and Reduce

Consider the following two documents:

```
doc1: data science data
doc2: science needs data
```

For the Word Count problem, the mapper produces:

$$(\text{word}, 1)$$

for every occurrence of a word.

1. Mapper Output

Write all intermediate key–value pairs produced by the mapper.
For example:

$$(\text{data}, 1)$$

Complete the table.

Document	Mapper Output
doc1	
doc2	

2. Shuffle and Sort

Group all intermediate values having the same key.
Complete:

$$\text{data} \rightarrow [\quad\quad\quad\quad]$$

$$\text{science} \rightarrow [\quad\quad\quad\quad]$$

$$\text{needs} \rightarrow [\quad\quad\quad\quad]$$

3. Reduce

The reducer calculates:

$$\text{count}(w) = \sum_i v_i.$$

Write the final output:

$$(\text{data}, \quad\quad)$$

$$(\text{science}, \quad\quad)$$

$$(\text{needs}, \quad\quad)$$

Answer:

1. What is the input key to the mapper in a document-processing MapReduce application?
2. What is the input value?
3. What key is produced by the Word Count mapper?
4. What does the shuffle and sort stage do?
5. Why must all intermediate values having the same key reach the same reducer?

Part 2: Implementing Word Count

Create a text file called:

`input.txt`

containing at least five lines of text.
For example:

```
Parallel computing can process large datasets
MapReduce processes data using map and reduce
Parallel systems can process data faster
Data can be distributed across many machines
MapReduce is designed for distributed processing
```

Mapper

Create:

`mapper.py`

The mapper should:

1. read input from standard input;
2. convert words to lowercase;
3. extract individual words;
4. output one pair

$$(\text{word}, 1)$$

for every word.

Use tab-separated output:

```
word    1
```

You may start with:

```
import sys
import re

for line in sys.stdin:

    line = line.strip().lower()

    words = re.findall(r"[a-z0-9']+", line)

    for word in words:

        # Output the intermediate key-value pair
        # TODO
```

Run the mapper:

```
python3 mapper.py < input.txt
```

Save the output:

```
python3 mapper.py < input.txt > map_output.txt
```

Inspect:

```
cat map_output.txt
```

Shuffle and Sort

MapReduce groups intermediate data according to the key.

For this exercise, simulate the shuffle and sort stage using:

```
sort
```

Run:

```
python3 mapper.py < input.txt | sort
```

Observe how equal keys become adjacent.

Answer:

1. Does `sort` calculate the word counts?
2. What useful property does sorting provide to the reducer?

Reducer

Create:

```
reducer.py
```

The reducer receives sorted intermediate pairs and calculates the total count for every word.

You may start with:

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:

    line = line.strip()

    word, count = line.split("\t")
    count = int(count)

    if word == current_word:

        # TODO: update count
        pass

    else:

        # TODO: output previous word if necessary

        # TODO: start counting a new word
        pass

# TODO: output the final word
```

Run the complete MapReduce pipeline:

```
python3 mapper.py < input.txt | sort | python3 reducer.py
```

Save the final result:

```
python3 mapper.py < input.txt | sort | \
python3 reducer.py > output.txt
```

Part 3: Follow the Data Flow

Choose the following input:

```
red blue red
blue green
red green blue
```

Write the data after each stage.

Map Output

After Shuffle and Sort

Final Reduce Output

Answer:

1. Does a mapper need to know the final number of occurrences of a word?
2. Does one mapper need the output of another mapper before it can process its own input?
3. Which stage brings together values generated by different mappers?
4. Is the reducer executed before all relevant mapper outputs become available?

Part 4: Combiner

A combiner performs local aggregation after mapping and before the global shuffle.

Suppose two mappers independently process:

```
Mapper 1 input:
data data science data

Mapper 2 input:
science data science
```

Without a combiner, Mapper 1 produces:

```
data    1
data    1
science 1
data    1
```

Instead, a combiner could locally produce:

```
data    3
science 1
```

Task

Create:

`combiner.py`

You may reuse the same aggregation logic as the reducer.
Create two input files:

`part1.txt`
`part2.txt`

Run:

```
python3 mapper.py < part1.txt | sort | \
python3 combiner.py > combined1.txt

python3 mapper.py < part2.txt | sort | \
python3 combiner.py > combined2.txt
```

Combine both local outputs:

```
cat combined1.txt combined2.txt | sort | \
python3 reducer.py
```

Compare the number of records before and after using the combiner:

```
wc -l map_output.txt
wc -l combined1.txt
```

Answer:

1. Does a combiner change the final Word Count result?
2. Does it reduce the number of intermediate records that may need to be transferred?
3. Why can reducing intermediate data reduce network traffic?
4. Why can a summation operation safely use a combiner?

Part 5: Partitioning

Assume that the system has two reducers:

$$R_0, \qquad R_1.$$

The partitioner determines which reducer receives each intermediate key.
For this exercise, use the simple rule:

$$\text{partition}(k) = \left(\sum_{c \in k} \text{ASCII}(c) \right) \bmod 2.$$

For example, all occurrences of the same word must produce the same partition number.
Calculate the partition for:

Key	Reducer
data	
science	
parallel	
computer	
hadoop	
reduce	

Answer:

1. Can the same intermediate key be sent to two different reducers in a normal per-key reduction?
2. Can two different keys be sent to the same reducer?
3. Why is partitioning necessary when several reducers are used?
4. Does partitioning guarantee that all reducers receive exactly the same amount of work?

Part 6: Inverted Index

An inverted index maps every word to the documents containing that word.

Create the file:

`documents.txt`

containing:

```
doc1    parallel computing uses many processors
doc2    mapreduce processes large data
doc3    parallel data processing is useful
doc4    computing systems process data
```

Each line contains:

$$(\text{document ID}, \text{text}).$$

Create:

`index_mapper.py`

The mapper should emit:

$$(\text{word}, \text{document ID})$$

for every word.

For example:

```
parallel    doc1
computing   doc1
```

Then create:

`index_reducer.py`

The reducer should produce output similar to:

```
computing   doc1,doc4
data        doc2,doc3,doc4
parallel    doc1,doc3
```

Do not print the same document ID more than once for the same word.
Run:

```
python3 index_mapper.py < documents.txt | sort | \
python3 index_reducer.py
```

Questions

1. What is the intermediate key in this application?
2. What is the intermediate value?
3. Why must all occurrences of one word be sent to the same reducer?
4. How is this application different from Word Count?

Part 7: MapReduce Runtime

For each operation below, state whether it is mainly the responsibility of the **programmer** or the **MapReduce runtime**.

Operation	Programmer / Runtime
Define the mapper	
Define the reducer	
Schedule map tasks	
Group intermediate values by key	
Shuffle intermediate data	
Detect failed workers	
Restart failed tasks	
Choose what the application computes	

Answer:

1. Why is hiding scheduling and communication details useful to the programmer?
2. What happens between the map and reduce phases?
3. Why can intermediate data movement become expensive?

Part 8: Data Locality and Distributed Storage

Answer the following questions briefly.

1. Why does a MapReduce scheduler try to execute a map task close to the physical location of its input data?
2. What is meant by “moving computation to the data”?
3. What is the purpose of a distributed file system in a MapReduce environment?
4. What is the role of the HDFS NameNode?

5. What is the role of an HDFS DataNode?
6. Does the NameNode normally transfer the actual file contents between the client and DataNodes?
7. Why are file blocks replicated on several machines?
8. How can replication improve fault tolerance?

Part 9: Short Discussion

Answer each question in 2–4 sentences.

1. What is the difference between the Map phase and the Reduce phase?
2. What is the purpose of the shuffle and sort stage?
3. Why are key–value pairs central to the MapReduce model?
4. What is the purpose of a combiner?
5. What is the difference between a combiner and a partitioner?
6. Why is Word Count naturally parallelizable?
7. Give one example of a problem that is well suited to MapReduce.
8. Give one example of a problem that would be difficult to express efficiently as a single MapReduce job.

Submission

Submit:

1. `mapper.py`;
2. `reducer.py`;
3. `combiner.py`;
4. `index_mapper.py`;
5. `index_reducer.py`;
6. `input.txt` and `documents.txt`;
7. the final Word Count output;
8. the final inverted-index output;
9. completed tables;
10. answers to all questions.

All submitted Python programs should run using Python 3.